



Electron Control System

Electron Architecture

Scenario - Lite S/X

Scenario - Lite Ka

Spaceport

Electron and spaceport extensions

Electron - Scheduler Camriere and ...

Electron - Scheduler brokkr extensi...

Electron - ingest api extension

Electron - NAOS extension

Electron - Receptor gathering exte...

Spaceport - Monitor API with Cosm...

Spaceport - Lite Monitor and Contr...

Spaceport - SpaceNintendo GUI

MEOS Control Systems

Architecture

Falcon

Hypermatter

Girodyne

Observability

Grafana

OpenTelemetry

Deployment

GN Products Deployment Services

Deployment with AWX

Deployment to Openstack

Quick Guide - Inventory and Deplo...

How to make api key in KOGNI for Ele...

Vault

Spaceport Extension - lite_monitor_control_api

The lite_monitor_control_api enables KSAT Lite customers to monitor and control the backend. This extension is only available for Kratos backend such as the qRadio, qFep, qRX and qDRA, SpectralNet and Orbital ACU. as well as the Orbital ACU.

The specific containers in this extension its worth noting are

[Bacchetta](#) - The "internal" control api

[Spaceport Control API](#) - Customer facing api

[Monitoring API](#) - Customer facing api

[Maestro scraper](#) - scrapes Kratos backend for metrics during a contact.

[Maestro dispatcher](#)

[Maestro azure publisher](#) - Publish scraped data to the Azure ADX -> to Grafana.

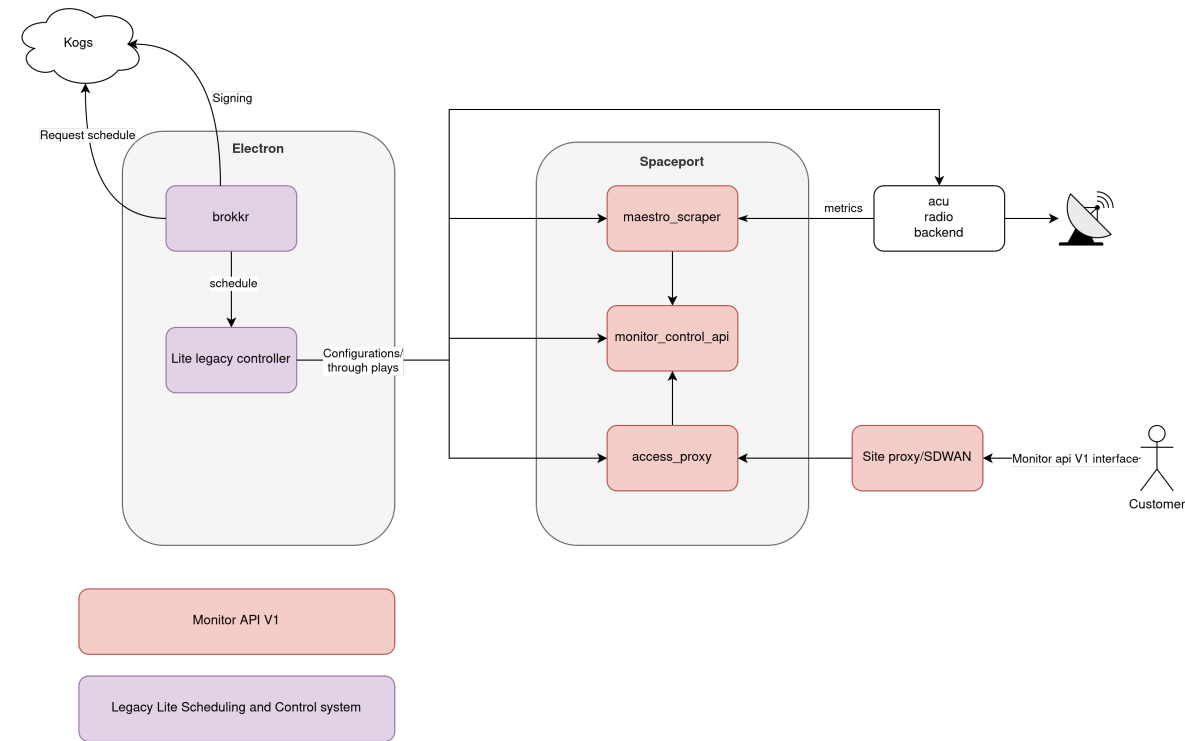
The image below shows a stripped down representation of the deployment. For monitoring and control api to work, Electron must be configured with legacy_controller, and the antenna must be represented in Kogni. A Site proxy must be set up by Marius Flage that routes the customer to spaceport (access proxy).

[Spaceport Extension - lite...](#)

Deployment

Additional variables nee...

How to test



Deployment

Additional variables needed for deployment in the spaceport inventory

VARIABLE	DESCRIPTION
iot_hub_connection_string	A connection string managed in azure. Should be defined in the antenna vault
azure_proxy	true / false Describes if the azure publisher needs to be deployed with a proxy to azure

How to test

Useful link for testing control api:

[Internal documentation](#). If you want to test using site proxy, use the [following link](#)
[Customer facing documentation](#)

Electron Control System

Electron Architecture

Scenario - Lite S/X

Scenario - Lite Ka

Spaceport

Electron and spaceport extensions

Electron - Scheduler Camriere and ...

Electron - Scheduler brokkr extensi...

Electron - ingest api extension

Electron - NAOS extension

Electron - Receptor gathering exte...

Spaceport - Monitor API with Cosm...

Spaceport - Lite Monitor and Contr...

Spaceport - SpaceNintendo GUI

MEOS Control Systems

Architecture

Falcon

Hypermatter

Girodyne

Observability

Grafana

OpenTelemetry

Deployment

GN Products Deployment Services

Deployment with AWX

Deployment to Openstack

Quick Guide - Inventory and Deplo...

How to make api key in KOGNI for Ele...

Vault

Michael.Johansen@ksat.no

If you want to test the monitor api, you can use the following script on the spaceport (already installed on most spaceports under `/tools/websocket_check.py`):

```
import json
import asyncio
import websockets

API_KEY="ENCJfW84HVlN7UgtCeQZNMinXozlzy+ecFckiHQfYE+72LA7"

async def consume(message):
    result = json.loads(message)
    print ("Received '%s'" % result)

async def run():
    headers = {"Authorization": f"apikey {API_KEY}"}
    async with websockets.connect("ws://localhost:80/monitor/websocket/receive", extra_headers=headers) as ws:
        while True:
            try:
                message = await ws.recv()
                await consume(message)
            except:
                break

if __name__ == "__main__":
    loop = asyncio.get_event_loop()
    loop.run_until_complete(run())
```